

NAME - PAVNI GULYANI

BATCH-16

SAPID- 590033879

Experiment - 6

Implementation of Iterative and Decision-Making Constructs in C

ALGORITHM

1. START
2. initialize integer variables for accumulators: $TC = 0$ (Total Credits) and $TGC = 0$ (Total Grade Credits).
3. Declare float variable: $SGPA$ and character variable: $grade$.
4. Prompt the user and read $global_id$, $semester$, and the number of courses n .
5. Initialize loop from $i = 1$ to n .
6. Inside the loop (for each course):
 7. Prompt and read course code, credit, and grade.
 8. Determine the Grade Point (GP) using conditional logic:
 9. If $grade == 'A+'$, set $GP = 10$
 10. Else If $grade == 'A'$, set $GP = 9$
 11. Else If $grade == 'B+'$, set $GP = 8$
 12. Else If $grade == 'B'$, set $GP = 7$
 13. Else If $grade == 'C+'$, set $GP = 6$

14. Else If grade == 'C', set GP = 5
15. Else If grade == 'D', set GP = 4
16. Else If grade == 'F', set GP = 0
17. Update Total Credits: $TC = TC + \text{credit}$
18. Update Total Grade Credits: $TGC = TGC + (GP * \text{credit})$
19. Calculate the final SGPA using the formula: $SGPA = (\text{float})TGC / TC$
20. Print the student's global_id, semester, and the calculated SGPA.
21. STOP

TEST-CASE TABLE

Input in the program	Expected output	Actual output	Pass/fail
Enter Global ID: 93874646 Enter Semester: 1 Enter number of courses: 4 - Course 1 - -- Enter Course Code: 101 Enter Credit: 4 Enter Grade (O, A, B, C, D, E, P, F): A	=====SGPA===== Global ID : 93874646 Semester : 1 Number of Courses: 4 Total Credits : 11 SGPA : 9.36	=====SGPA===== Global ID : 93874646 Semester : 1 Number of Courses: 4 Total Credits : 11 SGPA : 9.36	pass

--- Course 2 --- Enter Course Code: 203 Enter Credit: 2 Enter Grade (O, A, B, C, D, E, P, F): O -- Course 3 --- Enter Course Code: 1490 Enter Credit: 3 Enter Grade (O, A, B, C, D, E, P, F): A			
--	--	--	--

C-Program:

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int global_id, semester, code, n;
```

```
    int TC = 0, TGC = 0;
```

```
    int GP, credit;
```

```
    float SGPA;
```

```
    char grade;
```

```

// Switched back to char since there are no '+' signs

/* Get student details */

printf("Enter Global ID: ");

scanf("%d", &global_id);

printf("Enter Semester: ");

scanf("%d", &semester);

printf("Enter number of courses: ");

scanf("%d", &n);

/* Repeat for each course */

for (int i = 1; i <= n; i++)

{

    printf("\n--- Course %d ---\n", i);

    printf("Enter Course Code: ");

    scanf("%d", &gcode);

    printf("Enter Credit: ");

    scanf("%d", &gcredit);

    printf("Enter Grade (O, A, B, C, D, E, P, F): ");

    // The space before %c is CRITICAL to prevent the terminal from skipping!

    scanf(" %c", &grade);

    /* Find Grade Point using the new conversion scale */

    if (grade == 'O' || grade == 'o')

    {

        GP = 10;

    }

    else if (grade == 'A' || grade == 'a')

```

```
{  
    GP = 9;  
}  
else if (grade == 'B' || grade == 'b')  
{  
    GP = 8;  
}  
else if (grade == 'C' || grade == 'c')  
{  
    GP = 7;  
}  
else if (grade == 'D' || grade == 'd')  
{  
    GP = 6;  
}  
else if (grade == 'E' || grade == 'e')  
{  
    GP = 5;  
}  
else if (grade == 'P' || grade == 'p')  
{  
    GP = 4;  
}  
else if (grade == 'F' || grade == 'f')  
{
```

```

GP = 0;
}
else
{
printf("Invalid grade entered!\n");
return 0;

// Exits the program if a bad grade is typed
}

/* Calculate Total Grade Credits */
TGC = TGC + (GP * credit);

/* Calculate Total Credits */
TC = TC + credit;
}

/* Calculate SGPA */
if (TC != 0)
{
SGPA = (float)TGC / TC;
}
else
{
SGPA = 0;
}

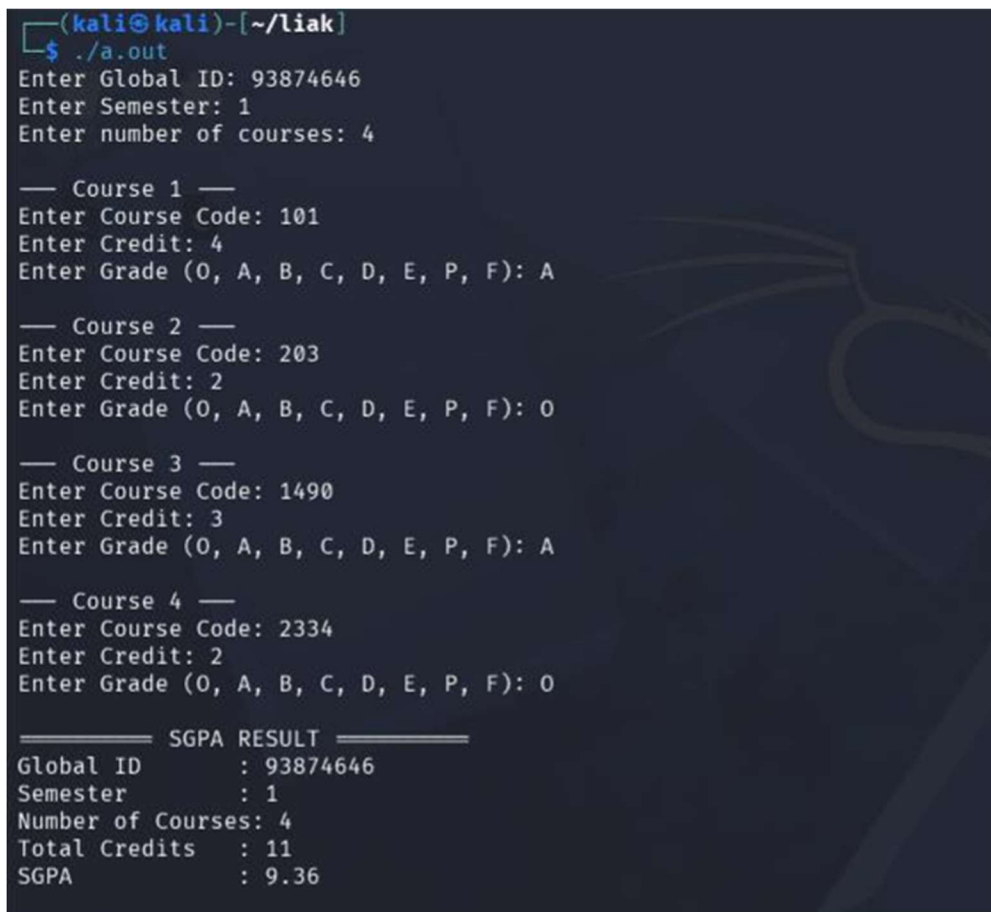
/* Display result */
printf("\n===== SGPA RESULT =====\n");
printf("Semester    : %d\n", semester);
printf("Number of Courses: %d\n", n);

```

```
printf("Total Credits : %d\n", TC);  
printf("SGPA: %.2f\n", sgpa);  
return 0;  
  
}  
  
}
```

Compilation and Execution: Use cc to compile the code, if there is no error

Use ./a.out to run the program



```
(kali㉿kali)-[~/liak]  
$ ./a.out  
Enter Global ID: 93874646  
Enter Semester: 1  
Enter number of courses: 4  
  
— Course 1 —  
Enter Course Code: 101  
Enter Credit: 4  
Enter Grade (0, A, B, C, D, E, P, F): A  
  
— Course 2 —  
Enter Course Code: 203  
Enter Credit: 2  
Enter Grade (0, A, B, C, D, E, P, F): O  
  
— Course 3 —  
Enter Course Code: 1490  
Enter Credit: 3  
Enter Grade (0, A, B, C, D, E, P, F): A  
  
— Course 4 —  
Enter Course Code: 2334  
Enter Credit: 2  
Enter Grade (0, A, B, C, D, E, P, F): O  
  
===== SGPA RESULT =====  
Global ID      : 93874646  
Semester       : 1  
Number of Courses: 4  
Total Credits  : 11  
SGPA           : 9.36
```

Question 2:

Calculate the monthly electricity bill of Domestic/residential consumers in Uttarakhand

ALGORITHM

1. START

2. Define variables int(connection_no, load, past_meter_read, current_meter_read, actual_unit)

2. Define variable float(fppca, dues, amt_payable)

3. Use the formula $\text{actual_units} = (\text{current_meter_read} - \text{past_meter_read})$

4. Get input for all the variable defined that need input from the user

5. if $\text{actual_unit} \leq 100$ ($\text{fppca} = 3.65 * \text{actual_units}$)

6. if $\text{actual_unit} \leq 200$ ($\text{fppca} = 100 * 3.65$)

($\text{fppca} = \text{fppca} + (\text{actual_unit} - 100) * 5.25$)

7. elif $\text{actual_unit} \leq 400$

($\text{fppca} = 100 * 3.65$)

($\text{fppca} = \text{fppca} + (\text{actual_units} - 100) * 5.25$)

($\text{fppca} = \text{fppca} + (\text{actual_units} - 200) * 7.15$)

8. elif $\text{actual_units} \geq 401$

($\text{fppca} = 100 * 3.65$)

($\text{fppca} = \text{fppca} + (\text{actual_units} - 100) * 5.25$)

($\text{fppca} = \text{fppca} + (\text{actual_units} - 200) * 7.15$)

($\text{fppca} = \text{fppca} + (\text{actual_units} - 400) * 7.80$)

9. Get input for load

10. if-elif ladder for load

(upto 1kw ₹5/kw)

(1-4 kw 85/kw)

(above 4 100/kw)

11. calculate the amount payable

(amount_payable=fccpa+dues+fixed_charge_duty_charge)

12. print past_meter_read, present_meter_read, Electricity_duty, amount_payable, connection_no, Outstanding dues, actual_units, fixed_charge, fppca

13. STOP

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int connection_no, load, past_meter_read, present_meter_read;
```

```
int actual_unit, fixed_charge;
```

```
float energy_charge, duty_charge, fppca_rate, fppca, dues, amt_payable;
```

```
printf("Enter Connection number: ");
```

```
scanf("%d", &connection_no);
```

```
printf("Enter Sanctioned load in kw: ");
```

```
scanf("%d", &load);
```

```
printf("Enter Previous meter reading: ");
```

```
scanf("%d", &past_meter_read);
```

```
printf("Enter Present meter reading: ");
```

```
scanf("%d", &present_meter_read);
```

```
printf("Enter FPPCA rate (in RS per unit): ");
```

```
scanf("%f", &fppca_rate);
```

```
printf("Enter Outstanding dues: ");
```

```
scanf("%f", &dues);
```

```
/* Calculate actual units consumed */
```

```
actual_unit = present_meter_read - past_meter_read;
```

```
/* Electricity duty */
```

```
duty_charge = 0.15 * actual_unit;
```

```
/* FPPCA */
```

```
fppca = fppca_rate * actual_unit;
```

```
/* Energy charge calculation */
```

```
if (actual_unit <= 100)
```

```
{
```

```
    energy_charge = actual_unit * 3.65;
```

```
}
```

```
else if (actual_unit <= 200)
```

```
{
```

```
    energy_charge = (100 * 3.65) +
```

```

        ((actual_unit - 100) * 5.25);
    }
    else if (actual_unit <= 400)
    {
        energy_charge = (100 * 3.65) +
            (100 * 5.25) +
            ((actual_unit - 200) * 7.15);
    }
    else
    {
        energy_charge = (100 * 3.65) +
            (100 * 5.25) +
            (200 * 7.15) +
            ((actual_unit - 400) * 7.80);
    }

```

/* Fixed charge calculation */

```

if (load <= 1)
{
    fixed_charge = load * 75;
}
else if (load <= 4)
{
    fixed_charge = load * 85;
}

```

```
else
{
    fixed_charge = load * 100;
}

/* Calculate final payable amount */
amt_payable = energy_charge + fixed_charge +
    duty_charge + fppca + dues;

/* Display bill details */
printf("\nConnection number: %d\n", connection_no);
printf("Sanctioned load: %d kW\n", load);
printf("Previous meter reading: %d\n", past_meter_read);
printf("Present meter reading: %d\n", present_meter_read);
printf("Actual units consumed: %d\n", actual_unit);

printf("Energy charge: Rs. %.2f\n", energy_charge);
printf("Fixed charge: Rs. %.2f\n", (float)fixed_charge);
printf("Electricity duty: Rs. %.2f\n", duty_charge);
printf("FPPCA: Rs. %.2f\n", fppca);
printf("Outstanding dues: Rs. %.2f\n", dues);
printf("Final amount payable: Rs. %.2f\n", amt_payable);

return 0;
}
```

Compilation and Execution:

use the cc command to compile the code and get the respective output

```
(kali㉿kali)-[~/liak]
$ cc electric.c

(kali㉿kali)-[~/liak]
$ ./a.out
Enter Connection number: 654327
Enter Sanctioned load in kW: 6
Enter Previous meter reading: 4520
Enter Present meter reading: 5000
Enter FPPCA rate (in Rs per unit): 0.15
Enter Outstanding dues: 129.5

Connection number: 654327
Sanctioned load: 6 kW
Previous meter reading: 4520
Present meter reading: 5000
Actual units consumed: 480
Energy charge: Rs. 2944.00
Fixed charge: Rs. 600.00
Electricity duty: Rs. 72.00
FPPCA: Rs. 72.00
Outstanding dues: Rs. 129.50
Final amount payable: Rs. 3817.50
```